École nationale des ponts et chaussées

IMI Department Project - 2025-2026

Under the supervision of : Vincent Leclère

Artificial Intelligence for a Board Game

Progress Report, April 2026

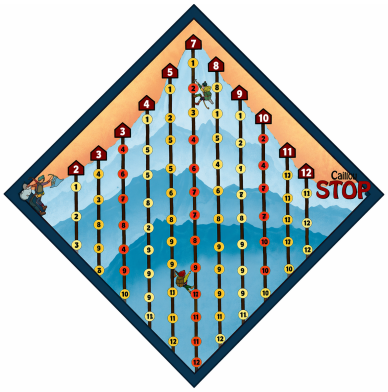


Figure 1 – Game board of Can't Stop

This report studies the optimization of the board game Can't Stop, a push-your-luck dice game in which players repeatedly roll four dice to advance on numbered tracks, while deciding at each turn whether to continue and risk losing their progress or to stop and secure their gains.

We proceed by increasing complexity :

— Section 1 introduces a simplified one-dimensional version of the problem, with a single column and Bernoulli steps, to develop the core dynamic programming framework.
— Section 2 verifies the probability distribution of dice sums used in the full game.
— Section 3 extends the model to three columns, which is the key setting of Can't Stop.
— Section 4 provides the general multi-column formulation.
— Section 6 presents known heuristic strategies from the literature.

1 One-dimensional case

As a first step, we consider a simplified version of the game involving a single column of height N. At each step, the player rolls and succeeds with probability p, earning one point. If the step fails, all progress accumulated during the current turn is lost. The player must decide at each step whether to *stop* (and secure the current score) or *continue* (and risk losing everything).

1.1 Stochastic model

Let $(X_i)_{1\leq i\leq N}$ be a sequence of independent Bernoulli random variables with parameter p :

$$\mathbb{P}(X_i = 1) = p, \qquad \mathbb{P}(X_i = 0) = 1 - p.$$

We denote by S_i the temporary score (i.e., the progress in the current turn) after step i. The process evolves according to

$$S_0 = 0$$

and for all $i \in \{0, \dots, N-1\}$,

$$S_{i+1} = \begin{cases} S_i + 1 & \text{if } X_{i+1} = 1, \\ 0 & \text{if } X_{i+1} = 0. \end{cases}$$

This defines a stochastic process which is a Markov chain with state space

$$\{0, 1, \dots, N\}.$$

The transition probabilities are

$$P(s, s+1) = p, \qquad P(s, 0) = 1 - p.$$

1.2 Optimal stopping problem

At each step, the player can choose to :
— continue the game,
— stop and secure the current score.
We denote by $\mathcal{T}$ the set of stopping times with respect to the natural filtration of the process. The objective is to maximize the expected final gain :

$$\max_{\tau\in\mathcal{T}} \mathbb{E}[S_\tau].$$

1.3 Dynamic Programming Formulation

We define the value function

$$u_t(s)$$

as the maximal expected gain when the current score is s and there are $N-t$ steps remaining. The terminal condition is

$$u_N(s) = s, \qquad \forall s \in \{0, \dots, N\}.$$

For $t < N$, the Bellman equation simplifies to

$$\boxed{u_t(s) = \max\big(s,\ p\, u_{t+1}(s+1)\big)}$$

where :

— the first term s corresponds to stopping and securing the current score,
— the second term $p\,u_{t+1}(s+1)$ corresponds to continuing the game, considering the probability of success p (failure leads to zero points).

1.4 Optimal strategy

The optimal value of the problem corresponds to the maximal expected gain when starting the game from the initial state. Since the game starts at time $t = 0$ with a score $S_0 = 0$, this value is

$$u_0(0).$$

This quantity represents the maximal expected score that the player can obtain when following an optimal strategy.

The optimal stopping rule is obtained by comparing the two possible actions in the Bellman equation. At each step, the player compares :

$$\text{Stop value} = s$$

and

$$\text{Continue value} = p\,u_{t+1}(s+1)$$

The player should stop whenever stopping yields a larger expected gain than continuing. Therefore, an optimal stopping time is

$$\tau^* = \inf\left\{t \geq 0 \ : \ s \geq p\,u_{t+1}(s+1)\right\}.$$

In practice, this strategy has a threshold structure : there exists a value $s^*(p)$ such that

$$\text{stop if } s \geq s^*(p), \qquad \text{continue otherwise.}$$

The threshold $s^*(p)$ depends on the success probability p and can be computed numerically using dynamic programming.

1.4.1 Threshold approximation for infinite horizon

To obtain an explicit expression for the optimal threshold $s^*(p)$, it is convenient to consider the *infinite-horizon* case, where the player can theoretically continue the game indefinitely. In this setting, the value function no longer depends explicitly on time, and we denote it simply by $u(s)$.

The Bellman equation for the infinite-horizon case reads

$$u(s) = \max\Big(s,\ p\,u(s+1)\Big),$$

where $u(0) = 0$ because the turn is lost if the first step fails.

The optimal stopping rule is determined by comparing the stop value and the continue value :

$$\text{Stop if } s \geq p\,u(s+1).$$

When the accumulated score s is sufficiently large (close to the optimal threshold), the contribution from future gains can be approximated by $u(s+1) \approx s+1$. Using this approximation, the indifference condition for stopping becomes

$$s^* \approx p\,(s^*+1).$$

Solving this inequality gives the explicit formula for the threshold :

$$s^*(p) = \frac{p}{1-p}.$$

Since in practice the score must be an integer, we define the actual threshold as the smallest integer satisfying the stopping condition :

$$s^*(p) = \left\lceil \frac{p}{1-p} \right\rceil.$$

This formula shows that the optimal threshold increases with the success probability p : the more likely a successful step, the higher the accumulated score s that it is worth risking for an additional point. Conversely, for small p, the player should stop early to avoid losing all accumulated points.

1.4.2 Example : $N = 10$, $p = 0.5$

Optimal policy $\pi(s)$:

s	$\pi(s)$
0	CONTINUE
1	CONTINUE
2	STOP
3	STOP
4	STOP
5	STOP
6	STOP
7	STOP
8	STOP
9	STOP
10	STOP

Expected values $u(s)$ under the optimal policy :

s	$u(s)$
0	0.5
1	1.0
2	2.0
3	3.0
4	4.0
5	5.0
6	6.0
7	7.0
8	8.0
9	9.0
10	10.0

This confirms the threshold structure of the optimal policy : the player continues when $s < 2$ and stops when $s \geq 2$.

1.4.3 Comparison of $s^*(p)$: formula vs numerical

The following table compares the theoretical threshold with the numerical results obtained via dynamic programming.

p	$s^*(p)$ formula	s^* numerical
0.1	1	1
0.2	1	1
0.3	1	1
0.4	1	1
0.5	1	2
0.6	2	2
0.7	3	3
0.8	5	5
0.9	10	10

We observe that the numerical thresholds are consistent with the theoretical prediction, especially for larger values of p. Small discrepancies may appear for intermediate values due to the finite horizon used in the numerical computation.

2 Verification of the Dice Sum Probabilities

Before tackling the full optimization problem, we verify that the dice mechanics of Can't Stop are correctly modeled. In the game, a player rolls four dice and forms pairs, where the sum of each pair determines which column can be advanced. With four dice, there are three possible pairings, giving up to three distinct column sums per roll.

In this section, we fix the attention on a single pair of dice and verify that the theoretical probability distribution of their sum matches the results of a Monte Carlo simulation. This validation step ensures that the probabilistic model used in the dynamic programming formulation is reliable.

2.1 Theoretical probabilities

The table below shows the theoretical probability p_k that at least one pair of dice sums to k, for $k = 2, \ldots, 12$:

k	Theoretical p_k
2	0.1319
3	0.2330
4	0.3557
5	0.4475
6	0.5610
7	0.6435
8	0.5610
9	0.4475
10	0.3557
11	0.2330
12	0.1319

These probabilities were obtained either analytically by enumerating all possible outcomes of two dice.

2.2 Simulation

We simulated $N_{\text{trials}} = 100{,}000$ trials of four dice and computed the empirical probabilities of obtaining at least one pair summing to k.

The simulated values are very close to the theoretical ones, as shown in the table below :

k	Theoretical p_k	Simulated p_k
2	0.1319	0.1316
3	0.2330	0.2340
4	0.3557	0.3584
5	0.4475	0.4483
6	0.5610	0.5564
7	0.6435	0.6445
8	0.5610	0.5620
9	0.4475	0.4460
10	0.3557	0.3556
11	0.2330	0.2320
12	0.1319	0.1326

2.3 Graphical comparison

Figure 2 shows a comparison between the theoretical and simulated probabilities.

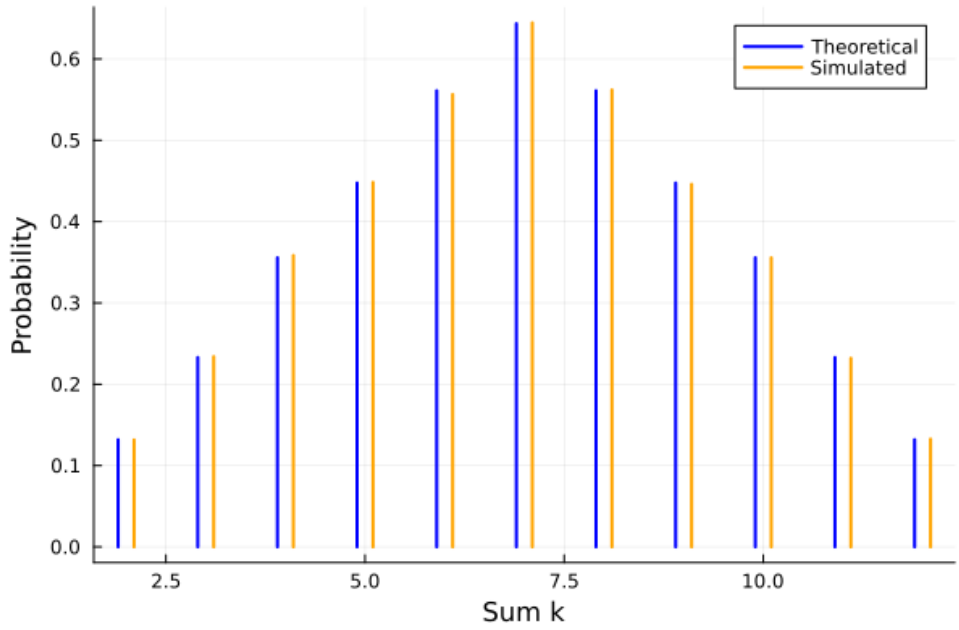


FIGURE 2 – Comparison of theoretical and simulated probabilities of sums k from four dice.

2.4 Interpretation

The table and figure confirm that the simulation reproduces the theoretical probabilities accurately.

3 Three-column problem

We now consider a simplified version of *Can't Stop* focusing on three fixed columns : 6, 7, and 8. These correspond to the most central and most frequently obtained sums when rolling two dice, making them particularly relevant for analysis.

In this setting, the player repeatedly rolls four dice and forms pairs to advance along these columns. As in the original game, each turn involves a sequence of decisions : after each roll, the player may either *stop* to secure the progress made during the turn, or *continue* and risk losing it entirely.

This simplified framework captures the core risk–reward trade-off of the game while remaining tractable. Our objective is to determine an optimal strategy that maximizes the expected gain.

3.1 Context

We consider a game where three columns (numbered 6, 7, and 8) are available to add dice according to the game's rules. Each column has a maximum height :

— N_6 for column 6,
— N_7 for column 7,
— N_8 for column 8.

The current state of the game is represented by a triplet (x, y, z), where :

— x is the height reached in column 6,
— y is the height reached in column 7,
— z is the height reached in column 8.

Let $\mathcal{C} = \{6, 7, 8\}$ be the three columns considered in this simplified model. A roll of four dice produces an outcome $\omega = (d_1, d_2, d_3, d_4) \in \{1, \dots, 6\}^4$. The outcome space is

$$\Omega = \{1, \dots, 6\}^4, \quad |\Omega| = 1296,$$

each outcome being equally likely with probability $p_\omega = 1/1296$

A roll of four dice can be paired in exactly three ways. We denote by

$$\mathcal{A}(\omega)$$

the set of these three possible pairings, where each pairing is a *multiset* of at most two column indices.

Example 1. *For $\omega = (2, 3, 4, 5)$ we have*

$$\mathcal{A}(\omega) = \big\{\{5, 9\}, \{6, 8\}, \{7, 7\}\big\}.$$

Example 2 (Doubles)**.** *For $\omega = (3, 3, 5, 5)$ we have*

$$\mathcal{A}(\omega) = \big\{\{6, 10\}, \{8, 8\}\big\}.$$

3.2 Game

— **State** : (x, y, z) represents the current heights of the columns.
— **Choice** : At each step, choose to continue or stop to maximize the total gain.
— **Constraints** : Heights cannot exceed N_6, N_7, N_8.
— **Goal** : Maximize the cumulative gain over the time horizon T.

3.3 Bellman Equation

The goal is to maximize the total gain $V_t(x, y, z)$ at each step t. This gain is defined as the maximum between :

1. The immediate gain if we stop playing : $x + y + z$,
2. The expected future gain if we continue playing : $V_{t+1,\text{continue}}(x, y, z)$.

The Bellman equation is written as :

$$V_t(x, y, z) = \max\left(x + y + z, V_{t+1,\text{continue}}(x, y, z)\right)$$

3.4 General Dynamic Programming Formulation

To formalize the problem in a rigorous way, we first recall the standard framework of finite-horizon stochastic control and dynamic programming.

General Dynamic Programming Framework

We consider a finite-horizon stochastic control problem. The objective is to maximize the expected total reward :

$$\mathbb{E}\left[\sum_{t=0}^{T-1} g_t(X_t, U_t) + K(X_T)\right],$$

where X_t is the state at time t and U_t is the control (action).
The value function satisfies the standard Bellman equations :

$$V_T(x) = K(x),$$

where $K(x)$ is the terminal reward at the final state X_T, and

$$V_t(x) = \max_{U_t \in \mathcal{A}(x)} \mathbb{E}\Big[g_t(x, U_t) + V_{t+1}(X_{t+1})\Big],$$

where $\mathcal{A}(x)$ denotes the set of admissible actions at state x and $g_t(x, U_t)$ is the immediate reward at stage t.

3.5 Explicit Bellman Equation

In our 3-column model, the expectation can be written explicitly as a sum over all possible dice outcomes $\omega \in \Omega$:

$$\begin{aligned}\mathbb{E}\big[g_t(x, U_t) + V_{t+1}(X_{t+1})\big] = \sum_{\omega\in\Omega} p_\omega \Big[&\mathbf{1}_{\mathcal{A}(\omega,x,y,z)=\emptyset}\, V_{t+1}(x, y, z) \\ &+ \mathbf{1}_{\mathcal{A}(\omega,x,y,z)\neq\emptyset} \max_{C\in\mathcal{A}(\omega,x,y,z)} \big(g_t(x, y, z, C) + V_{t+1}(x', y', z')\big)\Big]\end{aligned} \tag{1}$$

The set of admissible actions depends on the current state (x, y, z) :

$$\mathcal{A}(\omega, x, y, z) = \Big\{C \in \mathcal{A}(\omega) \;\Big|\; (x < N_6 \text{ if } 6 \in C),\ (y < N_7 \text{ if } 7 \in C),\ (z < N_8 \text{ if } 8 \in C)\Big\}.$$

This set automatically enforces that finished columns cannot be advanced. In particular, a *bust* occurs when $\mathcal{A}(\omega, x, y, z) = \emptyset$.

For a chosen action C (a multiset), the next state is

$$\begin{aligned} x' &= \min\big(x + \text{mult}(6, C), N_6\big), \\ y' &= \min\big(y + \text{mult}(7, C), N_7\big), \\ z' &= \min\big(z + \text{mult}(8, C), N_8\big), \end{aligned} \tag{2}$$

where $\mathrm{mult}(c, C)$ denotes the multiplicity of column c in the multiset C.

Here :
— $C \in \mathcal{A}(\omega, x, y, z)$ is the action chosen optimally for outcome ω,
— p_ω is the probability of outcome ω,
— $g_t(x, y, z, C)$ is the immediate reward for action C.

The full Bellman equation then compares the value of stopping and continuing :

$$V_t(x,y,z) = \max\Big(\text{stop value},\ \sum_{\omega\in\Omega} p_\omega \Big[\mathbf{1}_{\mathcal{A}(\omega,x,y,z)=\emptyset}\, V_{t+1}(x,y,z) + \mathbf{1}_{\mathcal{A}(\omega,x,y,z)\neq\emptyset} \max_{C\in\mathcal{A}(\omega,x,y,z)} \big(g_t(x,y,z,C) + V_{t+1}(x',y',z')\big)\Big]\Big) \tag{3}$$

Reward Structures

1. **Reward at each advancement :** points are earned each time a column is advanced. Immediate reward and terminal reward :

$$g_t(x,y,z,C) = \mathrm{mult}(6,C) + \mathrm{mult}(7,C) + \mathrm{mult}(8,C), \quad K(x,y,z) = 0$$

$$V_T(x,y,z) = 0$$

Stop value : the current score accumulated so far on each column :

$$\text{stop value} = x + y + z$$

2. **Reward only at column completion :** points are earned only when reaching the top of a column. Immediate reward and terminal reward :

$$g_t(x,y,z,C) = 0, \quad K(x,y,z) = \mathbf{1}_{x=N_6} + \mathbf{1}_{y=N_7} + \mathbf{1}_{z=N_8}$$

$$V_T(x,y,z) = K(x,y,z)$$

Stop value : the points obtained by securing all completed columns :

$$\text{stop value} = K(x,y,z)$$

3.6 Minimization of the number of rounds

We now adopt a dual formulation of the problem, shifting from reward maximization to a cost-minimization perspective. Instead of accumulating gains, we seek to minimize the expected number of rounds required to complete all three columns.

In this setting, each round is associated with a unit cost, and the player's decisions are driven by the objective of reaching the terminal state in the shortest expected time.

We denote by

$$V(x,y,z)$$

the expected number of remaining rounds needed to complete the game starting from the secured state (x, y, z).

During a turn, with temporary progress (x', y', z'), the value function satisfies

$$V(x,y,z;x',y',z') = \min\big(\text{stop value},\ \text{continue value}\big),$$

where

$$\text{stop value} = 1 + V(x', y', z')$$

and

$$\begin{aligned}\text{continue value} = \sum_{\omega\in\Omega} p_\omega \Big[& \mathbf{1}_{\mathcal{A}(\omega,x,y,z)=\emptyset}\big(1 + V(x,y,z)\big) \\ & + \mathbf{1}_{\mathcal{A}(\omega,x,y,z)\neq\emptyset} \min_{C\in\mathcal{A}(\omega,x,y,z)} V(x'', y'', z'')\Big]\end{aligned} \tag{4}$$

with the next temporary state given by

$$x'' = \min\big(x'+\text{mult}(6,C),\, N_6\big), \quad y'' = \min\big(y'+\text{mult}(7,C),\, N_7\big), \quad z'' = \min\big(z'+\text{mult}(8,C),\, N_8\big).$$

4 General multi-column formulation

We now generalize the three-column model to the full game of *Can't Stop*. The set of columns is $\mathcal{C} = \{2, 3, \dots, 12\}$, each column $c \in \mathcal{C}$ having a maximum height N_c. The definitions of Ω, $\mathcal{A}(\omega)$, $\mathcal{A}(\omega, x, x')$ and $\mathrm{mult}(c, C)$ introduced in Section 3 extend naturally to this general case.

The state of the game is described by two vectors :
— $x = (x_c)_{c\in\mathcal{C}}$: the *secured* progress in each column (updated only when the player stops) ;
— $x' = (x'_c)_{c\in\mathcal{C}}$: the *temporary* progress within the current turn, with $x'_0 = x$ and $x'_t \geq x$ for all t.

The objective is to minimize the expected number of turns needed to complete all columns, i.e., to reach $x_c = N_c$ for all $c \in \mathcal{C}$.

We denote by $V(x)$ the expected number of remaining rounds starting from the secured state x. During a turn we define the in-round value

$$\mathcal{V}(x, x') = \min(\text{stop}, \text{continue}),$$

where

$$\text{stop} = 1 + V(x')$$

and

$$\begin{aligned}\text{continue} = \sum_{\omega\in\Omega} p_\omega \Big[&\mathbf{1}_{\mathcal{A}(\omega,x,x')=\emptyset}\big(1 + V(x)\big) \\ &+ \mathbf{1}_{\mathcal{A}(\omega,x,x')\neq\emptyset} \min_{C\in\mathcal{A}(\omega,x,x')} \mathcal{V}(x, x''(C))\Big].\end{aligned} \tag{5}$$

State transition For each column $c \in \mathcal{C}$,

$$x''_c(C) = \min\big(x'_c + \mathrm{mult}(c, C),\ N_c\big),$$

where $\mathrm{mult}(c, C)$ is the multiplicity of c in the multiset C.

Active columns

$$\mathcal{C}_{\text{act}}(x') = \{c \in \mathcal{C} \mid x'_c > x_c\}.$$

Admissible actions

$$\mathcal{A}(\omega, x, x') = \Big\{C \in \mathcal{A}(\omega) \;\Big|\; \mathrm{supp}(C) \subseteq \{c \in \mathcal{C} : x'_c < N_c\} \text{ and } \big|\mathcal{C}_{\text{act}}(x') \cup \mathrm{supp}(C)\big| \leq 3\Big\},$$

where $\mathrm{supp}(C)$ denotes the support of the multiset C (i.e. the set of distinct columns appearing in C).

Bust condition

$$\mathcal{A}(\omega, x, x') = \emptyset \quad \Rightarrow \quad \text{the player busts and the round ends with no secured progress.}$$

5 Opening the columns

5.1 Back to one column

We consider a new problem : we have open a column $c \in \{2, 3, ..., 12\}$ whose height is N_c. We aim at estimating the minimal average number of rounds to reach the end of the column. We denote by p_{cc} the probability of getting 2 c, p_c the probability of getting exactly one c, p_{bust} the probability of bust, and p_{other} the probability of not having a c but having the possibility of playing on another open column.

Let us emphasize on the fact that : p_{bust} and p_{other} heavily depend on the other open columns. However, we may do not know them. Let us firstly assume for the seek of simplicity that they are known.

We also notice that these probabilities induce a choice (for example, it is possible with the four dices to make a couple with two c or with just one c). So, we assume that we always make the "best choice" : if two c are available, we neglect the case that we may sometimes make just one c. We thus define probabilities in this order : $p_{cc}, p_c, p_{bust}, p_{other}$ with disjoint unions each time.

We introduce the Bellman function $V^c_{x_0}(x)$ which represents the average number of rounds to reach the top of the column c when we are in x starting from a protected position x_0.

We still have the following form for the Bellman equation :

$$V^c_{x_0}(x) = \min(\text{stop}, \text{continue})$$

Where

$$\text{stop} = 1 + V_x(x)$$

And

$$\text{continue} = p_{bust}V^c_{x_0}(x_0) + p_cV^c_{x_0}(x+1) + p_{cc}V^c_{x_0}(x+2) + p_{other}V^c_{x_0}(x)$$

The following boundaries conditions hold :

$$V^c_{N_c}(N_c) = 1, V^c_{N_c}(x) = 0 \text{ otherwise}$$

How to estimate p_{bust} ? (we then recover p_{other} by using the fact that the 4 probabilities are summing to 1)

If two columns are already open, there is nothing to do, p_{bust} can be exactly computed. Otherwise, we could do a mean of all the probabilities of bust. However it is not optimal at all (we never open 4 if 2 and 3 are already open, but in that way we would consider 4 as an option...). Another idea would be to consider every configuration where column c is open (there are $\binom{10}{2} = 45$ cases), we use the exact resolution and we end by doing the average between all these cases : it gives an approximated average number of rounds for each column, we always try to open the columns with a lower "forecast" number of rounds.

6 Heuristic Strategies for *Can't Stop*, from paper "A Generalized Heuristic for Can't Stop"

The players has 2 types of markers to track his progress : white markers (for progress of his current action) and colored markers (for his full progress).

6.1 The Rule of 28

The **Rule of 28** is a foundational heuristic used to decide both **when to stop a turn** and **how to pair the dice**. It was created by players.

When to stop : A *progress value* is computed from the current neutral markers. If this total is ≥ 28, the player should stop their turn.

For each column i, define its contribution by

$$(s_i + 1)\big(|7 - i| + 1\big),$$

where s_i is the number of spaces the neutral marker has advanced in column i.

The following difficulty adjustments are then added :

$$\begin{cases} +2 & \text{if all three neutral markers are in odd columns,} \\ -2 & \text{if all three neutral markers are in even columns,} \\ +4 & \text{if all markers are high } (\geq 7) \text{ or all low } (\leq 7). \end{cases}$$

How to pair the dice : Among all possible dice pairings, choose the one with the highest *move value*, defined by

$$\sum_i \left(p_i(6 - |7 - i|) - 6 \cdot \text{marker}(i)\right),$$

where :

— p_i is the progress in column i,

— $\text{marker}(i) = 1$ if a new neutral marker is placed in column i,

— $\text{marker}(i) = 0$ otherwise.

This formula favors the middle columns and penalizes the use of additional neutral markers.

6.2 Constant Weights Heuristic

It comes from an optimization of the parameters of the Rule of 28 using a genetic algorithm, obtaining a strategy that performs about 18% better.

Progress weights : The columns values are constant. Columns 2 and 12 receive the highest value :

$$w_2^{\text{prog}} = w_{12}^{\text{prog}} = 7,$$

while the middle column 7 receives the lowest value :

$$w_7^{\text{prog}} = 1.$$

Move weights : There is a strong preference for columns 2 and 12 :

$$w_2^{\text{move}} = w_{12}^{\text{move}} = 7,$$

and also for column 8 :

$$w_8^{\text{move}} = 4.$$

Odd columns are generally avoided for moves, except for column 7.

Difficulty scores : The additional scores are :

$$\text{Odds} = +7, \qquad \text{Evens} = +1, \qquad \text{Highs} = +6, \qquad \text{Lows} = +5.$$

Other parameters :

$$\text{Marker penalty} = 6, \qquad \text{Stop threshold} = 29.$$

6.3 Linear Weights Heuristic

This is the most advanced heuristic presented in the paper. Unlike the constant-weight model, it allows the move weights to vary linearly depending on how far a marker has progressed in a column.

It achieved an average score of

$$9.05 \text{ turns to win,}$$

compared to

$$10.74 \text{ turns}$$

for the Rule of 28.

Move weights : The move value is computed using

$$v = \left\lfloor m_i \cdot \frac{j}{l_i} + b_i \right\rfloor,$$

where :

— j is the current position in the column,
— l_i is the length of column i,
— m_i and b_i are learned parameters for column i.

This heuristic behaves similarly to the Constant Weights champion :

— it assigns very high value to the outer columns 2 and 12,
— it gives low value to most odd columns.

Stop threshold

$$\text{Stop threshold} = 24.$$

When linear weights plays against constant weights, constant weights wins 54% of times.

6.4 General Principles for a Simulator

If the goal is to implement general-purpose rules in a simulator, the paper suggests the following principles.

Outer column priority : Evolved strategies strongly favor columns 2 and 12. Although these columns are harder to roll, they are short, so progress in them is especially valuable.

Neutral marker conservation : Applying a penalty of about 4 to 6 points for each newly used neutral marker helps reduce the risk of losing all progress during a turn.

Risk assessment : Certain column configurations, such as having all markers in odd columns, are inherently riskier. A good simulator should include this type of positional risk when deciding whether to roll again.